

CHAPTER 1 · SUB-CHAPTER 1.4

Collections

Map Implementations

[HashMap](#) · [LinkedHashMap](#) · [TreeMap](#) · [ConcurrentHashMap](#) · [EnumMap](#) · [WeakHashMap](#) · [IdentityHashMap](#)

[52+ Q&A](#) · [Deep Internals](#) · [Coding Exercises](#) · [Cheat Sheet](#)

[Java Backend Architecture Interview Prep](#)

Section 1 — Map Interface

Why Map does not extend Collection

The `Map<K,V>` interface does **not** extend `Collection<E>` because the conceptual model is fundamentally different. A `Collection` contains individual elements; a `Map` associates key-value pairs. A `Map` cannot sensibly implement methods like `add(element)`, `iterator()`, or `contains(element)` without ambiguity (add a key? a value? a pair?). The designers deliberately kept them separate hierarchies. `Map` provides its own iteration via `entrySet()`, `keySet()`, and `values()` which return `Collection` views.

Interview tip: 'Map does not extend Collection' is a classic question. Explain the conceptual mismatch clearly: Collections hold elements; Maps hold associations. Also note that `Map.Entry` is an inner interface providing typed key-value pair access.

Core Map Methods

- **put(K key, V value)** — associates key with value; returns previous value or null
- **get(Object key)** — returns value for key, or null if absent
- **getOrDefault(Object key, V defaultValue)** — returns value or defaultValue if key absent
- **putIfAbsent(K key, V value)** — puts only if key not already present; returns existing or null
- **computeIfAbsent(K key, Function<K,V> mappingFn)** — computes and stores value only if absent (lazy)
- **computeIfPresent(K key, BiFunction<K,V,V> remappingFn)** — recomputes value only if key present
- **compute(K key, BiFunction<K,V,V> remappingFn)** — always invokes fn; null result removes the key
- **merge(K key, V value, BiFunction<V,V,V> remappingFn)** — if absent puts value; if present merges
- **forEach(BiConsumer<K,V> action)** — iterates all entries
- **entrySet()** — returns `Set<Map.Entry<K,V>>` (preferred iteration method)
- **keySet()** — returns `Set<K>` (key view)
- **values()** — returns `Collection<V>` (value view, may contain duplicates)
- **containsKey(Object key) / containsValue(Object value)**
- **remove(Object key) / remove(Object key, Object value)** (conditional remove)
- **replace(K key, V value) / replace(K key, V oldVal, V newVal)**
- **replaceAll(BiFunction<K,V,V> fn)** — transforms all values in-place
- **size() / isEmpty() / clear()**

Map.Entry<K,V> Interface

`Map.Entry` represents a single key-value association. Retrieved via `entrySet()`. Methods: `getKey()`, `getValue()`, `setValue(V value)`. Iterating via `entrySet()` is more efficient than iterating `keySet()` and calling `get()` for each key — avoids a second hash lookup per entry.

```
// Preferred entry iteration
for (Map.Entry<String, Integer> entry : map.entrySet()) {
    System.out.println(entry.getKey() + " -> " + entry.getValue());
}

// Java 8+ forEach (equivalent, cleaner)
map.forEach((key, value) -> System.out.println(key + " -> " + value));

// Stream-based
map.entrySet().stream()
    .filter(e -> e.getValue() > 10)
    .forEach(e -> process(e.getKey(), e.getValue()));
```

Java 9+ Immutable Map Factory Methods

`Map.of(k1,v1, k2,v2, ...)` creates an unmodifiable map (up to 10 entries). `Map.ofEntries(Map.entry(k,v), ...)` handles any number of entries. `Map.copyOf(existingMap)` creates an unmodifiable copy. These maps throw `UnsupportedOperationException` on mutation, do NOT allow null keys or values, and have no guaranteed iteration order (in

fact, order is intentionally randomized each JVM run to prevent hash flooding).

```
// Java 9+ immutable maps
Map<String, Integer> m = Map.of("one", 1, "two", 2, "three", 3);
Map<String, Integer> m2 = Map.ofEntries(
    Map.entry("alpha", 1),
    Map.entry("beta", 2),
    Map.entry("gamma", 3)
);
Map<String, Integer> copy = Map.copyOf(mutableMap); // unmodifiable snapshot
```

putIfAbsent vs computeIfAbsent

putIfAbsent(key, value) — the value is eagerly evaluated before the call even if the key already exists. Useful when value construction is cheap. Returns the existing value (or null if newly put). **computeIfAbsent(key, fn)** — the mapping function is only invoked if the key is absent (lazy). Critical for expensive-to-construct values or when you want to avoid unnecessary object creation. Returns the existing or newly computed value. Common pattern: building multi-maps (`computeIfAbsent(key, k -> new ArrayList<>()).add(item)`).

```
// putIfAbsent – value constructed eagerly (list always created)
map.putIfAbsent("key", new ArrayList<>()); // new ArrayList even if key exists!
// computeIfAbsent – value constructed lazily (only when key absent)
map.computeIfAbsent("key", k -> new ArrayList<>()).add("item"); // efficient
// merge – powerful for accumulation
map.merge("word", 1, Integer::sum); // if absent: put 1; if present: sum
```

Section 2 — HashMap Deep Dive

Internal Structure

`HashMap<K,V>` is backed by **`Node<K,V>[] table`** — an array of buckets. Each Node stores (hash, key, value, next). The default initial capacity is 16 (must be power-of-2). The hash function applies **spreading**: `hash = key.hashCode() ^ (h >> 16)` — XORing the high 16 bits into the low 16 bits ensures high-bit information participates in bucket selection even when capacity is small. The bucket index is computed as `index = hash & (capacity - 1)` (equivalent to modulo but faster; requires power-of-2 capacity).

```
// Simplified HashMap internals (from JDK source)
static final int hash(Object key) {
    int h;
    return (key == null) ? 0 : (h = key.hashCode()) ^ (h >> 16);
}

// Bucket index
int index = hash & (table.length - 1); // power-of-2 trick

// Node structure
static class Node<K,V> {
    final int hash;
    final K key;
    V value;
    Node<K,V> next; // linked list for collision chain
}
```

Collision Handling: Linked List to Tree

- Collisions form a **linked list** within the bucket (chaining)
- When a bin reaches **`TREEIFY_THRESHOLD = 8`** AND `table.length >= MIN_TREEIFY_CAPACITY = 64`, the linked list is converted to a **Red-Black tree** (`TreeNode`)
- If `table.length < 64` at treeify time, `HashMap` instead **resizes** (doubles capacity) — cheaper than treeifying small tables
- On removal: when tree bin drops below **`UNTREEIFY_THRESHOLD = 6`**, tree reverts to linked list
- Treeified bins give $O(\log n)$ worst-case lookups vs $O(n)$ for linked list — protection against hash DoS attacks

Load Factor and Resizing

The default **load factor is 0.75** — a threshold representing the fraction of capacity at which resize occurs. When `size > capacity * 0.75`, the table is **doubled in capacity** and all entries are rehashed ($O(n)$ operation). The 0.75 value is a carefully chosen tradeoff: lower load factor (e.g. 0.5) wastes memory; higher (e.g. 1.0) increases collision probability. With a good hash function and 0.75 load, each bucket averages <1 entry. To avoid resize overhead when the expected size is known, pre-size with the formula: **`new HashMap<>(expectedSize / 0.75 + 1)`**.

Pre-sizing tip: `HashMap(expectedSize)` does NOT account for load factor. Use `HashMap(expectedSize/0.75 + 1)` or Guava's `Maps.newHashMapWithExpectedSize(n)` to avoid the resize. For 100 expected entries: `new HashMap<>(134)` avoids any resize.

Null Key Handling

`HashMap` allows **exactly one null key** (hashed as 0 — bucket 0) and any number of null values. `LinkedHashMap` inherits the same behavior. In contrast, `ConcurrentHashMap` and `Hashtable` forbid null keys and values. `TreeMap` forbids null keys (unless a `Comparator` that handles null is provided) since `Comparable.compareTo(null)` throws `NullPointerException`.

Modern HashMap Operations (Java 8+)

```
Map<String, Integer> freq = new HashMap<>();
// merge: if absent put value; if present apply function
```

```
freq.merge("apple", 1, Integer::sum);
freq.merge("apple", 1, Integer::sum); // freq["apple"] = 2
// compute: always invokes function; null result removes key
freq.compute("apple", (k, v) -> v == null ? 1 : v + 1);
// computeIfAbsent: invoke only if key absent
Map<String, List<String>> groups = new HashMap<>();
groups.computeIfAbsent("fruits", k -> new ArrayList<>()).add("apple");
// computeIfPresent: invoke only if key present
freq.computeIfPresent("apple", (k, v) -> v > 5 ? null : v + 1); // null removes
// replaceAll: transform all values in-place
freq.replaceAll((k, v) -> v * 2); // double all counts
// getOrDefault: avoid null check
int count = freq.getOrDefault("banana", 0);
// putIfAbsent: put only if not present
freq.putIfAbsent("cherry", 0);
// forEach: clean iteration
freq.forEach((word, count2) -> System.out.println(word + ": " + count2));
```

Section 3 — LinkedHashMap Deep Dive

Internal Structure

LinkedHashMap extends HashMap but adds **before** and **after** pointers to each Entry, forming a **doubly-linked list** that runs through all entries. This list maintains iteration order. Bucket array structure is identical to HashMap; the linked list is orthogonal to the bucket chains.

```
// LinkedHashMap Entry extends HashMap.Node
static class Entry<K,V> extends HashMap.Node<K,V> {
    Entry<K,V> before, after; // doubly-linked list pointers
}
// head -> entry1 <-> entry2 <-> entry3 -> tail (iteration order)
```

Two Ordering Modes

- **Insertion-order (default)** — entries iterate in the order they were first inserted. Subsequent puts of an existing key do NOT change its position.
- **Access-order** — constructor: `new LinkedHashMap<>(16, 0.75f, true)` — every `get()`, `put()`, `getOrDefault()`, or `compute()` that accesses an entry moves it to the END of the list (most-recently-used end). This enables building an LRU (Least Recently Used) cache by removing from the head.

LRU Cache Implementation

```
import java.util.*;

public class LRUCache<K, V> extends LinkedHashMap<K, V> {
    private final int capacity;

    public LRUCache(int capacity) {
        // accessOrder=true: get() moves entry to end of list
        super(capacity, 0.75f, true);
        this.capacity = capacity;
    }

    // Called after every put/putAll; remove eldest when over capacity
    @Override
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > capacity;
    }

    public V getOrNull(K key) {
        return getOrDefault(key, null);
    }
}

// Usage
LRUCache<Integer, String> cache = new LRUCache<>(3);
cache.put(1, "one");
cache.put(2, "two");
cache.put(3, "three");
cache.get(1); // access 1 -> moves to end; order: 2,3,1
cache.put(4, "four"); // evicts eldest (2); order: 3,1,4
System.out.println(cache.containsKey(2)); // false - evicted
```

Use Cases

- **Ordered output** — maintaining insertion order for JSON-like serialization
- **LRU cache** — access-order mode with `removeEldestEntry` override
- **Predictable iteration** — when the order of map entries matters (config, properties)
- **Deduplication with order preservation** — insert items; later duplicates don't change order

Warning: Access-order LinkedHashMap is NOT thread-safe. Concurrent reads via `get()` MODIFY the list. You must synchronize externally: `Collections.synchronizedMap(new LinkedHashMap<>(...))` — but note that iteration still requires explicit synchronization on the map object.

Section 4 — TreeMap Deep Dive

Red-Black Tree Backing

`TreeMap<K,V>` is backed by a **Red-Black tree** — a self-balancing BST with $O(\log n)$ guaranteed time for `get`, `put`, `remove`, and `containsKey`. Keys must implement **`Comparable<K>`** OR a **`Comparator<K>`** must be provided at construction time. Attempting to insert a non-Comparable key into a `TreeMap` without a `Comparator` throws **`ClassCastException`** at runtime.

```
// Natural ordering (keys must implement Comparable)
TreeMap<String, Integer> byName = new TreeMap<>();

// Custom ordering via Comparator
TreeMap<String, Integer> byLength = new TreeMap<>(
    Comparator.comparingInt(String::length).thenComparing(Comparator.naturalOrder())
);

// Reverse ordering
TreeMap<Integer, String> descending = new TreeMap<>(Comparator.reverseOrder());
```

NavigableMap Operations

`TreeMap` implements `NavigableMap<K,V>` (which extends `SortedMap`). This exposes powerful range and navigation operations, all $O(\log n)$:

- **`floorKey(k)`** — largest key $\leq k$, or null if none
- **`ceilingKey(k)`** — smallest key $\geq k$, or null if none
- **`lowerKey(k)`** — largest key strictly $< k$, or null if none
- **`higherKey(k)`** — smallest key strictly $> k$, or null if none
- **`firstKey()` / `lastKey()`** — min/max key
- **`firstEntry()` / `lastEntry()`** — min/max `Map.Entry`
- **`pollFirstEntry()` / `pollLastEntry()`** — retrieve AND remove min/max entry (deque-like)
- **`headMap(toKey)`** — live view of keys strictly $< toKey$
- **`tailMap(fromKey)`** — live view of keys $\geq fromKey$
- **`subMap(fromKey, fromInclusive, toKey, toInclusive)`** — live view with full inclusive/exclusive control
- **`descendingMap()`** — reverse-order view
- **`navigableKeySet()` / `descendingKeySet()`** — navigable key set views

```
TreeMap<Integer, String> map = new TreeMap<>();
map.put(1, "one"); map.put(3, "three"); map.put(5, "five");
map.put(7, "seven"); map.put(9, "nine");

System.out.println(map.floorKey(4)); // 3 (largest <= 4)
System.out.println(map.ceilingKey(4)); // 5 (smallest >= 4)
System.out.println(map.lowerKey(5)); // 3 (strictly < 5)
System.out.println(map.higherKey(5)); // 7 (strictly > 5)

// Live view: changes to subMap reflect in original and vice versa
SortedMap<Integer,String> sub = map.subMap(3, true, 7, false); // [3,7)
sub.put(4, "four"); // also appears in original map!

// Range query: all keys from 3 (inclusive) to 7 (exclusive)
map.subMap(3, 7).forEach((k,v) -> System.out.println(k + "=" + v));

// Poll: atomic remove-and-return
Map.Entry<Integer, String> lowest = map.pollFirstEntry();
```

`SortedMap` views (`headMap`, `tailMap`, `subMap`) are **LIVE** — they reflect ongoing modifications to the underlying `TreeMap`. Structural modifications through the view (`put/remove`) are reflected back to the original map. The view's range acts as a constraint: putting a key outside the range throws `IllegalArgumentException`.

Section 5 — ConcurrentHashMap Deep Dive

Java 8+ Internal Architecture

`ConcurrentHashMap<K,V>` in Java 8+ uses an array of **`Node<K,V>`** — similar to `HashMap`. Concurrency is achieved at the **bin (bucket) level**: (1) Empty bucket insert: uses **CAS (Compare-And-Swap)** — lock-free, no synchronization needed. (2) Non-empty bucket: **synchronized on the bin's head node** — only threads competing for the same bucket contend. This allows full concurrency across different buckets — $O(1)$ read performance with concurrent writers.

```
// Simplified ConcurrentHashMap put logic
final V putVal(K key, V value) {
    int hash = spread(key.hashCode());
    for (Node<K,V>[] tab = table;;) {
        Node<K,V> f; int n, i;
        if (tab == null || (n = tab.length) == 0)
            tab = initTable();
        else if ((f = tabAt(tab, i = (n-1) & hash)) == null) {
            // Empty bucket: CAS insert (lock-free)
            if (casTabAt(tab, i, null, new Node<>(hash, key, value, null)))
                break;
        } else {
            synchronized (f) { // Lock only on bin head
                // Insert into bin (linked list or tree)
            }
        }
    }
}
```

Why null Keys/Values Are Forbidden

`ConcurrentHashMap` intentionally **forbids null keys and null values**. The reason is subtle: if `get(key)` returns null, in a single-threaded `HashMap` you can call `containsKey(key)` to distinguish 'key mapped to null' from 'key absent'. In a concurrent context, between `get()` and `containsKey()` another thread could insert or remove the key — making the two-step check unreliable. By forbidding null values, the API guarantee is: `get()` returning null ALWAYS means key is absent, no ambiguity. This was a deliberate design choice by Doug Lea.

Atomic Operations

All compute-style operations are **atomic per key** — the function is invoked within the synchronized bin lock, preventing interleaving of check-then-act sequences:

```
ConcurrentHashMap<String, Integer> freq = new ConcurrentHashMap<>();
// Atomic increment – safe under concurrent access
freq.merge("key", 1, Integer::sum);
// Atomic computeIfAbsent – only one thread computes the value even under races
ConcurrentHashMap<String, List<String>> groups = new ConcurrentHashMap<>();
groups.computeIfAbsent("fruits", k -> new CopyOnWriteArrayList<>().add("apple"));
// Atomic compute – check-then-act as single operation
freq.compute("counter", (k, v) -> v == null ? 1 : v + 1);
// Parallel bulk operations (Java 8+)
long sum = freq.reduceValues(4, Integer::sum, 0L); // parallelism threshold=4
freq.forEachKey(4, System.out::println); // parallel forEach
freq.forEachEntry(4, e -> process(e));
```

`size()` and `mappingCount()`

size() returns an int — may be imprecise under concurrency (capped at Integer.MAX_VALUE). **mappingCount()** returns a long — preferred for large maps or when precision matters. Internally, ConcurrentHashMap uses a **LongAdder-based CounterCell array** to track size with minimal contention: each thread increments its own cell; the total is summed when requested. This is the same pattern as LongAdder/Striped64. Exact accuracy is not guaranteed under concurrent modification.

Comparison: ConcurrentHashMap vs Hashtable vs synchronizedMap

Feature	ConcurrentHashMap	Hashtable	synchronizedMap
Lock granularity	Per-bin (CAS or synchronized on bin head)	Entire table	Entire table (method-level)
Read concurrency	Full (lock-free reads)	Serialized	Serialized
Write concurrency	Concurrent on different bins	Serialized	Serialized
Null key/value	Neither allowed	Neither allowed	Depends on backing map
Iterator safety	Weakly consistent (no ConcModException)	Fail-fast	Fail-fast; iteration needs external lock
size() accuracy	Approximate under concurrency	Exact (locked)	Exact (locked)
Preferred use	High-concurrency maps	Legacy only	Quick wrap of existing map

Section 6 — EnumMap, WeakHashMap, IdentityHashMap

EnumMap

EnumMap<K extends Enum<K>, V> is backed by a plain **array indexed by the enum ordinal**. This gives O(1) performance for all operations with minimal memory overhead and no hashing required. It does NOT allow null keys (ordinal() on null throws NullPointerException). It DOES allow null values. Iteration order follows the **enum declaration order** (by ordinal). EnumMap is NOT thread-safe.

```
enum Day { MON, TUE, WED, THU, FRI, SAT, SUN }
EnumMap<Day, String> schedule = new EnumMap<>(Day.class);
schedule.put(Day.MON, "Stand-up 9am");
schedule.put(Day.FRI, "Retro 3pm");
// Iterates in enum declaration order: MON, TUE, ..., SUN
schedule.forEach((day, event) -> System.out.println(day + ": " + event));
// Always prefer EnumMap over HashMap when keys are an enum type
// EnumMap is faster, more memory-efficient, and provides ordered iteration
```

WeakHashMap

WeakHashMap<K,V> stores keys as **weak references** (java.lang.ref.WeakReference). When the GC determines that no strong references to a key exist elsewhere, the key (and its entry) become eligible for collection. The garbage-collected entries are not removed immediately — they are enqueued in an internal **ReferenceQueue** and purged lazily during the next structural operation (get, put, size, etc.) via expungeStaleEntries(). Used for **metadata caches** where the cache entry should not prevent the key from being GC'd (e.g., Class-to-metadata mappings, listener registrations).

```
// Typical use: class metadata cache that doesn't prevent class unloading
WeakHashMap<Class<?>, ClassMetadata> metaCache = new WeakHashMap<>();
metaCache.put(MyClass.class, new ClassMetadata(MyClass.class));
// When MyClass is unloaded (class loader GC'd), the entry is automatically removed
// WARNING: never use String literals or other strongly-referenced objects as keys –
// they will never be GC'd, defeating the purpose
// CAUTION: WeakHashMap is NOT thread-safe
// CAUTION: size() may return a stale count until stale entries are expunged
```

IdentityHashMap

IdentityHashMap<K,V> uses **reference equality (==)** instead of equals() for key comparison and **System.identityHashCode()** instead of hashCode(). Two distinct objects that are .equals() to each other are treated as DIFFERENT keys. Backed by a linear-probe hash table (not chained). Used in **object graph traversal** (cycle detection), **serialization** (tracking already-visited objects), and frameworks that track object identity rather than logical equality.

```
IdentityHashMap<String, Integer> idMap = new IdentityHashMap<>();
String s1 = new String("hello");
String s2 = new String("hello"); // equal content, different reference
idMap.put(s1, 1);
idMap.put(s2, 2);
System.out.println(idMap.size()); // 2! (s1 != s2 by reference)
System.out.println(idMap.get(s1)); // 1
System.out.println(idMap.get(s2)); // 2
// Use case: serialization visited-set to detect cycles
IdentityHashMap<Object, Boolean> visited = new IdentityHashMap<>();
visited.put(obj, Boolean.TRUE);
if (visited.containsKey(obj)) { /* cycle detected */ }
```

Section 7 — Map Operations Patterns

Frequency Count

```
String[] words = {"apple", "banana", "apple", "cherry", "banana", "apple"};
Map<String, Integer> freq = new HashMap<>();
// Method 1: merge (cleanest, Java 8+)
for (String w : words) freq.merge(w, 1, Integer::sum);
// Method 2: getOrDefault
for (String w : words) freq.put(w, freq.getOrDefault(w, 0) + 1);
// Method 3: computeIfAbsent + compute
for (String w : words) freq.compute(w, (k, v) -> v == null ? 1 : v + 1);
// Result: {apple=3, banana=2, cherry=1}
```

Group-By (Multi-Map)

```
// Group strings by first character
List<String> names = Arrays.asList("Alice", "Bob", "Anna", "Charlie", "Bruce");
Map<Character, List<String>> byLetter = new HashMap<>();
// computeIfAbsent is the idiomatic pattern
for (String name : names) {
    byLetter.computeIfAbsent(name.charAt(0), k -> new ArrayList<>()).add(name);
}
// {A=[Alice, Anna], B=[Bob, Bruce], C=[Charlie]}
// Stream equivalent
Map<Character, List<String>> byLetter2 = names.stream()
    .collect(Collectors.groupingBy(name -> name.charAt(0)));
```

Inverting a Map

```
Map<String, Integer> original = Map.of("one",1, "two",2, "three",3);
// Invert: values become keys, keys become values
// WARNING: if values are not unique, toMap will throw on duplicate keys
Map<Integer, String> inverted = original.entrySet().stream()
    .collect(Collectors.toMap(Map.Entry::getValue, Map.Entry::getKey));
// Handling duplicate values (keep last)
Map<Integer, String> invertedSafe = original.entrySet().stream()
    .collect(Collectors.toMap(
        Map.Entry::getValue,
        Map.Entry::getKey,
        (existing, replacement) -> replacement // merge fn for duplicates
    ));
```

getOrDefault vs containsKey + get

map.getOrDefault(key, def) performs a **single hash lookup**. **map.containsKey(key) + map.get(key)** performs **two hash lookups** — avoid this pattern. Exception: when you need to distinguish 'key maps to null' from 'key absent', you must use **containsKey()**.

```
// Efficient: single lookup
int count = freq.getOrDefault("word", 0);
// Inefficient: two lookups (double the hashing)
if (freq.containsKey("word")) {
    count = freq.get("word"); // second lookup!
}
// Only valid use of containsKey + get: null-value distinction
if (map.containsKey(key)) {
```

```
V val = map.get(key); // val may legitimately be null
process(val);
}
```

replaceAll and compute Patterns

```
// replaceAll: in-place value transformation
Map<String, Integer> prices = new HashMap<>(Map.of("a",100,"b",200,"c",300));
prices.replaceAll((item, price) -> (int)(price * 1.1)); // apply 10% increase

// compute: atomic check-then-update
// Use case: conditional decrement, never below 0
map.compute("counter", (k, v) -> {
    if (v == null || v <= 0) return null; // remove entry if 0
    return v - 1;
});

// merge: building running totals
Map<String, Double> totals = new HashMap<>();
for (Transaction t : transactions) {
    totals.merge(t.category(), t.amount(), Double::sum);
}
```

Section 8 — Choosing the Right Map (Decision Table)

Map Type	Thread Safe	Ordering	Null Key	Null Value	Complexity	Typical Use Case
HashMap	No	None (random)	Yes (1)	Yes	O(1) amortized	General purpose, fastest single-thread
LinkedHashMap	No	Insertion or Access	Yes (1)	Yes	O(1) amortized	LRU cache, ordered output, JSON
TreeMap	No	Sorted (natural/comparator)	No*	Yes	O(log n)	Range queries, sorted iteration, NavigableMap
ConcurrentHashMap	Yes (fine-grained)	None	No	No	O(1) amortized	High-concurrency shared map
Hashtable	Yes (coarse)	None	No	No	O(1) amortized	Legacy only — avoid
Collections.synchronizedMap	Yes (coarse)	Depends on backing	Depends	Depends	Same as backing	Quick thread-safe wrapper
EnumMap	No	Enum ordinal order	No	Yes	O(1) always	Enum keys — fastest/smallest map
WeakHashMap	No	None	No (weak)	Yes	O(1) amortized	Metadata caches, listener maps
IdentityHashMap	No	None	Yes	Yes	O(1) amortized	Object graph traversal, serialization
Map.of() / Map.copyOf()	Yes (immutable)	None (random)	No	No	O(1)	Immutable config/constants

* TreeMap with a null-tolerant Comparator can accept null keys.

Decision guide: (1) Enum keys? → EnumMap. (2) Need sort/range queries? → TreeMap. (3) LRU cache? → LinkedHashMap (access-order). (4) High concurrency? → ConcurrentHashMap. (5) Object identity (not equality)? → IdentityHashMap. (6) Weak-reference cache? → WeakHashMap. (7) Immutable constants? → Map.of(). (8) Everything else? → HashMap.

Q&A; — Standard Questions (Q1–Q35)

Q1. How does HashMap calculate the bucket index for a key?

HashMap first computes a spread hash: `hash = key.hashCode() ^ (key.hashCode() >>> 16)`. This XOR mixes the high 16 bits into the low 16 bits so that keys differing only in high bits land in different buckets even at small capacities. The bucket index is then: `index = hash & (capacity - 1)`. Since capacity is always a power of 2, this bitwise AND is equivalent to modulo but far cheaper. Example: `capacity=16, hash=0x12345678 → index = 8`.

Q2. What is the load factor in HashMap and why is 0.75 the default?

The load factor (default 0.75) is the threshold ratio (`size/capacity`) at which HashMap resizes. When size exceeds `capacity * loadFactor`, capacity doubles and all entries are rehashed. 0.75 is a carefully tuned tradeoff: too low (e.g. 0.5) wastes memory (many empty buckets); too high (e.g. 1.0) increases collisions and degrades performance. At 0.75 with a good hash function, the Poisson distribution predicts fewer than 0.5% of bins will have more than 8 entries (the treeify threshold), giving near- $O(1)$ average performance.

Q3. When does HashMap treeify a bin, and why?

HashMap treeifies a bin (converts a linked list to a Red-Black tree) when TWO conditions are met: (1) the bin length reaches `TREEIFY_THRESHOLD = 8`, AND (2) the table length is at least `MIN_TREEIFY_CAPACITY = 64`. If condition (2) is not met, HashMap resizes instead (doubling capacity redistributes entries and shortens chains more cheaply than treeification). Treeification protects against adversarial hash functions and hash flooding attacks by capping worst-case lookup at $O(\log n)$ instead of $O(n)$. When entries are removed and a bin shrinks below `UNTREEIFY_THRESHOLD = 6`, the tree converts back to a linked list to save memory.

Q4. Why does ConcurrentHashMap not allow null keys or values, while HashMap does?

HashMap is single-threaded: if `get(key)` returns null, you can safely call `containsKey(key)` to determine whether the key is absent or mapped to null. In a ConcurrentHashMap, between `get()` and `containsKey()` another thread could insert or remove the mapping, making the two-step check unreliable. By forbidding null values, the contract becomes unambiguous: `get()` returning null ALWAYS means the key is absent. Similarly, null keys are forbidden for symmetry and to avoid special-casing null in concurrent code paths. This was a deliberate design decision by Doug Lea.

Q5. How can you distinguish 'key maps to null' from 'key is absent' in a Map?

Use `map.containsKey(key)` to check presence, then `map.get(key)` to retrieve the (possibly null) value. Alternatively, `map.getOrDefault(key, SENTINEL)` with a distinct sentinel object. In ConcurrentHashMap this distinction is moot — null values are forbidden so `get()` returning null always means absent. In HashMap: `if (map.containsKey(key)) { V v = map.get(key); /* v may be null */ }`. Two lookups are required. The `getOrDefault` approach is simpler when null-value distinction is not needed.

Q6. What are the four ways to iterate over a Map's entries?

1) `entrySet()` loop: `for (Map.Entry e : map.entrySet()) { e.getKey(); e.getValue(); }` — PREFERRED; single hash lookup per entry. 2) `keySet()` loop: `for (K k : map.keySet()) { V v = map.get(k); }` — two lookups per entry; AVOID. 3) `values()` loop: `for (V v : map.values())` — only when keys are not needed. 4) `forEach BiConsumer`: `map.forEach((k,v) -> process(k,v))` — cleanest Java 8+ form. Equivalent performance to `entrySet()` loop. Never use `keySet()` + `get()` in a tight loop.

Q7. What is the difference between put() and putIfAbsent()?

`put(key, value)` ALWAYS inserts/replaces and returns the PREVIOUS value (or null if newly added). `putIfAbsent(key, value)` inserts ONLY if the key is not already present; returns the EXISTING value if present (so you know if it was a no-op), or null if newly inserted. Crucially, `putIfAbsent`'s value argument is evaluated EAGERLY before the call — even if the key already exists. For lazy/expensive value construction, use `computeIfAbsent` instead. In ConcurrentHashMap, `putIfAbsent` is also atomic (single synchronized operation).

Q8. How does `computeIfAbsent` differ from `putIfAbsent`?

`putIfAbsent(key, value)`: value is a pre-evaluated argument — created before the call regardless of whether the key is present. Returns existing value or null. `computeIfAbsent(key, mappingFunction)`: the Function is only INVOKED if the key is absent (lazy evaluation). Returns the existing or newly computed value — NOT null on new insertion. `computeIfAbsent` is ideal for multi-maps (`computeIfAbsent(key, k -> new ArrayList<>()).add(item)`) and expensive-to-construct values. It's also atomic in `ConcurrentHashMap` — even if two threads race, the function is called at most once per key.

Q9. Explain the LRU cache pattern with `LinkedHashMap`.

`LinkedHashMap`'s access-order mode (third constructor argument = true) moves each accessed entry to the END of the internal doubly-linked list. The FRONT holds the least-recently-used entry. Override `removeEldestEntry(eldest)` to return true when `size()` exceeds the capacity — `LinkedHashMap` will then automatically remove the eldest (LRU) entry after each put. This gives a complete LRU eviction policy in ~6 lines of code. The `get()` method also participates in ordering — accessing an entry promotes it. Time complexity: O(1) get and put (HashMap buckets for lookup + linked list pointer updates for ordering). Must synchronize externally for thread safety.

Q10. In `TreeMap`'s `NavigableMap`, what do `floor`, `ceiling`, `lower`, and `higher` return?

All methods return null if no qualifying key exists. `floorKey(k)`: largest key LESS THAN OR EQUAL to k (floor = <=). `ceilingKey(k)`: smallest key GREATER THAN OR EQUAL to k (ceiling = >=). `lowerKey(k)`: largest key STRICTLY LESS THAN k (excludes k itself). `higherKey(k)`: smallest key STRICTLY GREATER THAN k (excludes k itself). Memory aid: floor/ceiling include the key itself; lower/higher exclude it. `floorEntry/ceilingEntry/lowerEntry/higherEntry` return the full `Map.Entry`. All are O(log n).

Q11. Why is `EnumMap` always preferred over `HashMap` for enum keys?

`EnumMap` is backed by a plain array indexed by enum ordinal — no hashing required. This gives (1) O(1) time with zero collision probability; (2) lower memory — just an array vs `HashMap`'s array of linked lists/trees; (3) predictable iteration in enum declaration order; (4) better cache locality — array elements are contiguous in memory vs scattered `HashMap` nodes. The enum class is required at construction (`new EnumMap<>(Day.class)`) so JVM can size the array exactly. For flag-to-handler maps, day-to-schedule, etc., `EnumMap` is the correct choice.

Q12. What is `WeakHashMap` used for, and what happens to entries when the key is GC'd?

`WeakHashMap` stores keys as weak references — the presence of a key in the map does NOT prevent it from being garbage-collected. When the GC reclaims a key, the reference is enqueued in the map's internal `ReferenceQueue`. The map does NOT purge stale entries in real time — they are removed lazily during the next call to `get()`, `put()`, `size()`, or other structural operations via `expungeStaleEntries()`. Use cases: class-to-metadata caches (`ClassLoader/class` unloading), listener registries where the listener shouldn't be kept alive just by being in the map, and canonicalization caches. WARNING: String literals and other interned objects are never GC'd — using them as `WeakHashMap` keys defeats the purpose.

Q13. How can you make a Map thread-safe? What are the four options?

1) `ConcurrentHashMap`: best performance for high concurrency; fine-grained locking; atomic operations. 2) `Collections.synchronizedMap(map)`: wraps any map; coarse lock on every method; iteration requires external synchronization (`synchronized(map) { for (K k : map.keySet()) ... }`). 3) `Hashtable`: legacy class; coarse synchronization; avoid in new code. 4) Immutable map (`Map.of()` / `Map.copyOf()`): zero synchronization needed; best for read-only shared config. Rule: for mutable shared maps, always prefer `ConcurrentHashMap` unless you need sorted order (use `ConcurrentSkipListMap` for sorted concurrent access).

Q14. What is the difference between `ConcurrentHashMap.size()` and `mappingCount()`?

`size()` returns an int — imprecise under concurrent modification and capped at `Integer.MAX_VALUE`. `mappingCount()` returns a long — preferred for large maps and more semantically accurate. Both are APPROXIMATE under concurrent access: they sum up internal `LongAdder/CounterCell` arrays which may lag slightly behind actual concurrent puts/removes. Neither provides a strongly consistent count without external synchronization. For maps with more than 2 billion entries, `size()` would silently overflow — use `mappingCount()`. For most code, the distinction rarely matters practically.

Q15. What does Map.merge() do step by step?

merge(key, value, remappingFunction) works as follows: (1) If key is absent OR currently mapped to null: put(key, value) — inserts the provided value directly. (2) If key is present with a non-null existing value: invoke remappingFunction(existingValue, value). (3) If the function returns null: REMOVE the key from the map. (4) If the function returns non-null: update the key's mapping to the returned value. Classic use: frequency counting — merge(word, 1, Integer::sum). If 'word' is absent, maps to 1; if present, adds 1 to the existing count. The function is never called with null arguments.

Q16. How do you count element frequency using a Map? Show three approaches.

Approach 1 (merge): map.merge(word, 1, Integer::sum) — cleanest; handles absent/present in one call. Approach 2 (getOrDefault): map.put(word, map.getOrDefault(word, 0) + 1) — two operations; readable. Approach 3 (compute): map.compute(word, (k,v) -> v==null ? 1 : v+1) — explicit null check; more verbose. Approach 4 (computeIfAbsent then get): less idiomatic; use compute or merge instead. For concurrent scenarios: ConcurrentHashMap.merge(word, 1L, Long::sum) — atomic per key. For high-throughput concurrent counting: use a LongAdder per key (computeIfAbsent + LongAdder.increment()).

Q17. When would you use IdentityHashMap?

IdentityHashMap is appropriate when you need to track OBJECT IDENTITY (== reference equality) rather than logical equality (.equals()). Use cases: (1) Object graph traversal / cycle detection — recording visited nodes by reference prevents treating equal-but-distinct objects as the same node; (2) Serialization/deserialization — tracking already-serialized objects to handle object sharing; (3) Proxy/interception frameworks — associating metadata with specific object instances; (4) JVM internals and class transformation tools where identity matters. Avoid for general-purpose maps — unexpected behavior when two equal objects are treated as different keys.

Q18. What is the difference between map.forEach and entrySet().stream().forEach?

map.forEach(BiConsumer) directly iterates the internal structure without creating an intermediate Stream object — lower overhead, slightly faster for simple operations. entrySet().stream().forEach(...) creates a Stream pipeline: adds overhead but enables chaining operations (.filter(), .map(), .sorted(), .limit()) before the terminal forEach. Use map.forEach for straightforward iteration. Use entrySet().stream() when you need filtering, transformation, or collection of results. For parallel processing: entrySet().parallelStream() or ConcurrentHashMap's forEachEntry(parallelismThreshold, action) are more appropriate. Semantically: both are 'weakly consistent' on ConcurrentHashMap (see concurrent entries).

Q19. How do you get a sorted Map from an unsorted Map?

Option 1: Put all entries into a TreeMap — new TreeMap<>(existingMap). Iterates in key natural order. Option 2: TreeMap with custom Comparator — new TreeMap<>(comparator); putAll(existingMap). Option 3: Sort by value using Stream — existingMap.entrySet().stream().sorted(Map.Entry.comparingByValue()).collect(Collectors.toMap(e->e.getKey(), e->e.getValue(), (a,b)->a, LinkedHashMap::new)). The LinkedHashMap::new supplier preserves the sorted order in the collected result. Option 4: Sort entries into a List for one-time sorted iteration without a new map.

Q20. What happens if you modify a key's hashCode-affecting field while it's in a HashMap?

The key becomes 'lost' in the map. When originally put, the entry was placed in bucket hash(key1) & (n-1). After mutating the key, hash(key2) is different, so get(key) looks in bucket hash(key2) & (n-1) — which is a different bucket — and returns null. The entry still exists in the old bucket but is unreachable via normal get/remove operations. The map is now effectively corrupted for that entry. Fix: always use immutable objects (or immutable fields) as map keys. Java records and String are safe. If mutable objects must be keys, ensure the fields used in hashCode/equals are final and not modified after insertion.

Q21. How does HashMap handle a put() when two keys have the same hash?

Two keys with the same computed bucket index (hash collision) are placed in the same bin. HashMap traverses the bin's linked list comparing each node's key using == (reference equality) then key.equals(). If an exact key match is found, the value is updated (replaced) and the old value returned. If no match is found, a new Node is appended to the end of the linked list. In Java 8+, if the bin length reaches TREEIFY_THRESHOLD (8) and table length >= 64, the linked list is converted to a Red-Black tree for O(log n) traversal.

Q22. Explain the difference between HashMap's put() and replace() methods.

put(key, value): inserts if absent, updates if present — always changes the mapping. Returns previous value or null.
replace(key, value): updates ONLY IF key is already present — no-op if absent. Returns previous value or null (null could mean absent).
replace(key, oldValue, newValue): conditional replace — updates only if current value equals oldValue. Returns boolean. Use replace() when you want an update-only semantic without risking accidental insertion. In ConcurrentHashMap, replace(key, old, new) is atomic — useful for optimistic locking patterns.

Q23. What is the time complexity of TreeMap operations compared to HashMap?

HashMap: $O(1)$ amortized for get/put/remove/containsKey (assuming good hash distribution). Worst case $O(n)$ without treeification; $O(\log n)$ per bin after treeification. TreeMap: $O(\log n)$ GUARANTEED for all core operations — no amortized behavior, no resizing, no edge cases. The constant factor is higher than HashMap (pointer-heavy tree nodes vs array). TreeMap also provides $O(\log n)$ range operations (headMap, tailMap, subMap, floor, ceiling) which have no HashMap equivalent. For sorted-order iteration TreeMap is $O(n)$; HashMap is also $O(n)$ for iteration but in unpredictable order.

Q24. How does TreeMap determine key ordering and what happens with incompatible types?

TreeMap uses the Comparable interface (natural ordering) if no Comparator is provided. Keys must implement Comparable or a ClassCastException is thrown at runtime when the first comparison is needed (typically at put() time for the second key or later). With a Comparator, any key type is supported as long as the comparator handles it. The Comparator must be consistent with equals for correct Map semantics (if compare(a,b)=0, then a.equals(b) should be true). Inconsistent comparators (e.g., ordering by string length where 'ab' and 'cd' compare as equal) cause TreeMap to treat them as the same key — only one survives in the map.

Q25. What does Map.of() guarantee about immutability and ordering?

Map.of() returns an unmodifiable (immutable) Map — all mutation methods throw UnsupportedOperationException. The map does NOT guarantee any specific iteration order — it is implementation-defined and in practice intentionally randomized each JVM run (using a random seed at startup) to prevent hash flooding attacks. Null keys and null values are NOT allowed (NullPointerException). Duplicate keys in the varargs are detected and throw an IllegalArgumentException at construction time. Use Map.copyOf(map) to create an unmodifiable copy of an existing mutable map.

Q26. How does LinkedHashMap maintain insertion order?

Each entry in LinkedHashMap is a subclass of HashMap.Node that adds before and after pointers. These pointers form a doubly-linked list running through all entries in insertion order (or access order if that mode is enabled). The list has a head (oldest entry) and tail (newest entry). When a new entry is inserted, it is appended to the tail. When an entry is removed, the linked list pointers are updated to skip it. This overhead means each operation does slightly more pointer manipulation than HashMap, but time complexity remains $O(1)$ amortized.

Q27. Can a HashMap have duplicate keys?

No. A Map by contract maps each key to at most one value. Calling put(key, newValue) when key already exists REPLACES the old value and returns it — it does not create two entries. Key uniqueness is enforced via hashCode() and equals(): two keys are considered the same if key1.equals(key2) returns true (and their hash codes are equal, per the equals/hashCode contract). Note: if you violate the contract (hashCode() consistent but equals() inconsistent), you can end up with two 'equal' keys appearing to coexist — this is a bug, not a feature.

Q28. What is the initial capacity of HashMap and how does resize work?

Default initial capacity is 16 (must be a power of 2). Resize is triggered when $\text{size} > \text{capacity} * \text{loadFactor}$ (default: $\text{size} > 16 * 0.75 = 12$). On resize: capacity doubles ($16 \rightarrow 32$); a new backing array is allocated; all existing entries are rehashed and redistributed. Java 8+ optimization: due to the power-of-2 property, each entry either stays in its original bucket index or moves to $(\text{original_index} + \text{old_capacity})$. This means rehashing only checks one bit of the hash — an elegant $O(n)$ linear rehash with good cache performance.

Q29. How do you use Map as a cache with automatic expiry?

Java's built-in Map implementations do not support TTL (time-to-live) expiry natively. Options: (1) Guava Cache / Caffeine — production-grade caching libraries with TTL, max-size eviction, async loading, and statistics. (2) LinkedHashMap LRU — evicts by access frequency, not time. (3) ConcurrentHashMap + ScheduledExecutorService — store entries with expiry timestamps; schedule periodic cleanup that removes expired entries. (4) Java 11+ Caffeine (via Spring Cache). For production code, always prefer a dedicated caching library over hand-rolled expiry logic.

Q30. What is the difference between Hashtable and ConcurrentHashMap?

Hashtable: legacy class (Java 1.0); synchronizes EVERY method on the entire table — only one thread can access it at a time. No support for atomic compound operations. null keys/values forbidden. Extends Dictionary (not AbstractMap). Performance degrades significantly under contention. ConcurrentHashMap: modern design (Java 5+, redesigned in Java 8); per-bin synchronization with CAS for empty buckets — allows concurrent readers and multiple simultaneous writers on different bins. Provides atomic compound operations (compute, merge, computeIfAbsent). size() is approximate. Null keys/values forbidden. Implements ConcurrentMap interface. Always prefer ConcurrentHashMap.

Q31. What happens when you call get() on a TreeMap with an incompatible key type?

TreeMap.get(Object key) will throw ClassCastException at runtime if the key cannot be compared to the existing keys. The comparison is done via the Comparator (if provided) or by casting the key to Comparable and calling compareTo(). For example, putting Integer keys then calling get("string") on a TreeMap throws ClassCastException because String cannot be cast to Integer for comparison. This is different from HashMap where get() uses equals() and will simply return null for a non-matching type without throwing.

Q32. Explain how Map.compute() can be used to atomically update a value.

compute(key, BiFunction) is atomic in ConcurrentHashMap and equivalent to a synchronized read-then-write in HashMap. The function receives the current value (or null if absent) and returns the new value (returning null removes the key). This prevents the race condition in non-atomic code like: `V v = map.get(k); map.put(k, transform(v))`. Example atomic decrement that removes at zero: `map.compute(key, (k,v) -> (v == null || v <= 1) ? null : v - 1)`. In ConcurrentHashMap this runs under the bin lock — no other thread can interleave between the read and write for the same key.

Q33. What are the NavigableMap operations for range queries in TreeMap?

headMap(toKey): live view of all entries with key strictly < toKey. headMap(toKey, inclusive): Java 6+ overload with explicit inclusive flag. tailMap(fromKey): live view of all entries with key >= fromKey. tailMap(fromKey, inclusive): explicit inclusive flag. subMap(fromKey, toKey): live view of [fromKey, toKey) — from inclusive, to exclusive. subMap(fromKey, fromInclusive, toKey, toInclusive): full control over both bounds. All views are LIVE — modifications to the view are reflected in the original map and vice versa. Modifications through a view that violate the range constraint throw IllegalArgumentException.

Q34. How would you implement a thread-safe LRU cache using ConcurrentHashMap?

LinkedHashMap LRU is not thread-safe. For concurrent LRU: use Caffeine or Guava Cache (best option). DIY option: ConcurrentHashMap for O(1) get/put + ConcurrentLinkedDeque for access-order tracking. On get: look up in map; if found, update deque (remove then add to tail); if miss, load and put. On put: if size exceeds capacity, evict head of deque and remove from map. Atomic coordination requires care — use compute() to atomically check-and-insert. In practice: Caffeine's W-TinyLFU algorithm outperforms LRU for most workloads and is production-ready. Use `Caffeine.newBuilder().maximumSize(n).expireAfterWrite(ttl).build()` for production caches.

Q35. What is ConcurrentHashMap's forEachEntry() and how does parallelism work?

forEachEntry(long parallelismThreshold, Consumer<T> action) performs a bulk operation over all entries. If mappingCount() > parallelismThreshold, the operation executes in parallel using the ForkJoinPool.commonPool() via a divide-and-conquer approach over the table segments. If mappingCount() <= parallelismThreshold, it executes sequentially in the calling thread. Setting threshold to 1 forces maximum parallelism; setting to Long.MAX_VALUE forces sequential. Similar bulk operations: forEachKey, forEachValue, reduceEntries, searchEntries, mapReduceEntries. The action is weakly consistent — it may not see all modifications made during traversal.

Q&A; — Advanced Questions (Q36–Q52)

[ADV] Q36. Why does HashMap's hash spreading use XOR with $(h \gg 16)$? Provide a mathematical analysis.

HashMap's table index is $\text{hash} \& (\text{capacity} - 1)$. With small capacity (e.g. 16), only the lowest 4 bits determine the bucket. If all keys have the same low 4 bits in their hashCode (a common pattern for objects with sequential IDs, enum ordinals, etc.), they ALL land in the same bucket regardless of the upper bits — $O(n)$ degeneration. The spread: $h \wedge (h \gg 16)$ XORs the top 16 bits into the bottom 16 bits. Now even if the low bits are identical, the high bits (which differ between keys) influence the bucket index. Cost: one XOR and one unsigned right-shift — essentially free. It doesn't eliminate collisions but provides a cheap first defense against poor hashCode implementations. A full multiplicative hash (Knuth's) would be better but costlier — this is the pragmatic tradeoff.

[ADV] Q37. What are the five Red-Black tree invariants maintained by TreeMap, and how do they guarantee $O(\log n)$ height?

The five Red-Black tree invariants: (1) Every node is either red or black. (2) The root is black. (3) Every null leaf (sentinel) is black. (4) No two consecutive red nodes — if a node is red, both children are black. (5) All paths from any node to its descendant null leaves have the same number of black nodes (black-height is uniform). Invariant (5) guarantees the tree is 'approximately balanced': the longest path (alternating red-black) is at most 2x the shortest path (all black). So height $h \leq 2 * \log_2(n+1)$, making all operations $O(\log n)$. TreeMap's rebalancing after put/remove uses rotations and recolouring to restore these invariants in $O(\log n)$ time.

[ADV] Q38. How does ConcurrentHashMap's compute() guarantee atomicity? Explain how synchronized-on-bin-head prevents interleaving.

compute(key, fn) in ConcurrentHashMap works as follows: (1) compute the hash and find the bucket. (2) If bucket is empty, use CAS to atomically claim it. (3) If bucket is non-empty, acquire synchronized(binHead) — the intrinsic lock on the first node of the bin. Within this lock, compute() reads the current value, invokes the BiFunction, then writes the result (or removes if null). No other thread can acquire the same bin head lock concurrently — Java's intrinsic locks are mutually exclusive. This means the read-modify-write sequence is atomic with respect to all other operations on the same key/bucket. Different bins have different locks — full concurrency across bins is maintained. The function must not itself modify the map (would cause deadlock or corruption on the same key).

[ADV] Q39. Explain ConcurrentHashMap's size tracking via LongAdder/CounterCell arrays and why it avoids contention.

Naive size tracking with a single AtomicLong would cause contention: every put/remove would CAS-compete on the same cache line. ConcurrentHashMap uses a Striped64/LongAdder approach: an array of CounterCell objects, each on its own cache line (padded). Each thread maps to a cell by its thread-local probe (a pseudo-random index). On size increment: try CAS on the base count; if CAS fails (contention), fall back to the thread's assigned CounterCell and CAS there instead. Each cell is a padded long, so cells on different cache lines don't cause false sharing. Total size = base + sum(cells). mappingCount() sums all cells. This design scales linearly with core count.

[ADV] Q40. Why does HashMap resize by doubling to a power-of-2, and explain the elegant 'stay or move' property.

When capacity doubles from N to $2N$, the new bucket index for key k is: $\text{hash} \& (2N-1)$. The additional bit in the mask is bit $\log_2(N)$ of the hash. If this bit is 0: index stays the same (old index unchanged). If this bit is 1: index moves to (old index + N). This means each entry in a bin of old capacity N either stays in bucket i OR moves to bucket $i+N$ — never anywhere else. Java 8 exploits this: instead of full rehashing, it splits each bin into two sub-bins (stay-list and move-list) using a single pass through the chain in $O(\text{bin_length})$ time. The 'high bit' determines placement, and the split maintains insertion order within sub-bins. This makes resize $O(n)$ with excellent cache behavior and no random memory access per entry.

[ADV] Q41. Why does LinkedHashMap access-order mode modify the linked list on every get(), and what are the implications for concurrent use?

In access-order mode, `get()` calls `afterNodeAccess(e)`, which unlinks the accessed node from its current position in the doubly-linked list and relinks it at the tail (most-recently-used end). This is a WRITE operation on the linked list — three pointer updates (before/after of neighbors and the moved node). This means: (1) Two concurrent `get()` calls on access-order `LinkedHashMap` will race on the linked list pointers, causing data corruption. (2) Even read-only code (just `get()` calls) can corrupt the map under concurrency. You MUST synchronize externally even for `get()` on access-order `LinkedHashMap`. `Collections.synchronizedMap()` wraps only public methods — ensuring that `get()` acquires the lock. However, iteration still requires `synchronized(map){}` block.

[ADV] Q42. Explain WeakHashMap internals: the ReferenceQueue and how expungeStaleEntries works — why it's not real-time GC notification.

`WeakHashMap` wraps each key in a `WeakReference` registered with an internal `ReferenceQueue`. When the GC determines a key has no strong references, it enqueues the `WeakReference` in the `ReferenceQueue` — but does NOT call any `WeakHashMap` method. The `WeakHashMap` itself is unaware until the next structural operation (`get`, `put`, `size`, `resize`, etc.) calls `expungeStaleEntries()`, which drains the `ReferenceQueue` and removes the corresponding entries from the table. This means: (1) stale entries can remain visible to `size()` for an arbitrarily long time; (2) `get()` on a stale key correctly returns null because the `WeakReference.get()` returns null after GC; (3) the memory of the Entry (value object) is not freed until `expungeStaleEntries` runs. This is 'cooperative' cleanup, not real-time notification. A Phantom or Cleaner-based approach would be more timely but more complex.

[ADV] Q43. How does EnumMap forbid null keys? Explain the ordinal() NullPointerException mechanism.

`EnumMap` uses the enum's `ordinal()` method to compute the array index: `int index = key.ordinal()`. If key is null, calling `null.ordinal()` throws `NullPointerException` immediately — this is the enforcement mechanism. There is no explicit null check in `EnumMap`'s `put()` for most JVM implementations; the NPE is a natural consequence of calling a method on null. Compare this to `HashMap`, which explicitly checks for null and hashes null as 0. `EnumMap`'s array-based design makes explicit null handling less natural — and since the enum universe is finite and known at compile time, null key support would add complexity with minimal benefit. Values CAN be null (stored directly in the array slot; `EnumMap` uses a private Object UNMAPPED sentinel to distinguish absent from null).

[ADV] Q44. How does Java 9 Map.of() implementation use random seed shuffling to prevent hash flooding attacks?

`Map.of()` internally stores entries in a fixed-size array (for small maps) or an immutable hash table. The JDK implementation uses a random SALT (computed once at JVM startup via a static initializer using `ThreadLocalRandom` or a `SecureRandom` seed) in the hash computation. This means the iteration order and internal layout of `Map.of()` maps VARIES between JVM runs. An attacker who wants to construct a set of keys that all hash to the same bucket (hash flooding to create $O(n)$ lookup) cannot do so without knowing the JVM's salt — which is private and random. This is the same principle as hash randomization in Python dict and Go map. `HashMap` itself does not use this salt (it uses the key's own `hashCode()`), but treeification provides the same protection at the bin level.

[ADV] Q45. Why does HashMap.put() return the previous value, and what are the design implications for atomic check-and-set patterns?

`put(key, value)` returns the PREVIOUS value (or null if newly inserted). This return value enables one-lookup atomic-style patterns: `V prev = map.put(key, value); if (prev != null) { undo... }`. However, in multi-threaded code, this is NOT atomic. For atomic check-and-set: use `putIfAbsent(key, value)` which returns null if newly inserted (success) or the existing value (failure — key was already present). `ConcurrentHashMap.putIfAbsent` is a single atomic operation. Pattern: `V existing = concMap.putIfAbsent(key, computedValue); boolean inserted = (existing == null)`. Contrast: `map.put(k,v)` returning null does NOT distinguish 'newly inserted' from 'key was mapped to null' in a `HashMap` — use `putIfAbsent` for unambiguous first-insert detection.

[ADV] Q46. Why are SortedMap views (headMap/tailMap/subMap) live views backed by the original TreeMap? How do structural modifications reflect?

headMap(toKey) does NOT copy entries — it returns a SubMap object that holds a reference to the original TreeMap plus the range constraint [lo, hi). All reads/writes go through to the original tree. Structural modifications via the view (put within range) update the tree and are immediately visible to other views and the original map. Writes outside the range throw IllegalArgumentException. This design avoids the O(n) cost of copying a range and enables streaming patterns: tree.subMap(3, 7).values().stream().mapToInt(Integer::intValue).sum() iterates only the relevant tree range without copying. The live view semantics also mean that if the original map is modified externally, all existing views reflect the change immediately — no stale data. Contrast with List.subList() which has the same live-view semantics.

[ADV] Q47. Explain ConcurrentHashMap forEachEntry(parallelismThreshold, action) — how the threshold triggers ForkJoinPool parallelism.

forEachEntry(threshold, action) compares threshold against the map's current estimated size via mappingCount(). If mappingCount() >= threshold, the operation is submitted to ForkJoinPool.commonPool() using a RecursiveAction (Traverser) that recursively bisects the table into halves and processes each half independently. If mappingCount() < threshold (or threshold == Long.MAX_VALUE), the operation executes sequentially in the calling thread. The commonPool's parallelism level determines the maximum degree of concurrency. The threshold=1 forces max parallelism (one task per entry would be excessive — ForkJoinPool coalesces small tasks). Recommended threshold: table.size()/ForkJoinPool.getCommonPoolParallelism(). WARNING: actions must be non-interfering and stateless for correct parallel execution; the action should not modify the map.

[ADV] Q48. What happens internally when ConcurrentHashMap is resized? How does the 'transfer' work concurrently?

ConcurrentHashMap's resize (transfer) is fully concurrent. When resize is triggered: a new table of double the capacity is allocated. A ForwardingNode (special marker with hash = MOVED) is placed in each processed bin of the old table. Multiple threads participating in resize each claim a stripe of old table buckets (via CAS on transferIndex) and migrate entries from old to new table. A thread doing a put() that encounters a ForwardingNode calls helpTransfer() to assist with the ongoing resize before completing its own put() in the new table. Readers (get()) encountering a ForwardingNode follow its pointer to the new table. This cooperative resizing avoids blocking all threads during resize — the map remains partially accessible throughout the operation.

[ADV] Q49. How does the equals/hashCode contract affect HashMap correctness, and what happens when it's violated?

HashMap's contract relies on: (1) equal objects must have equal hashCodes; (2) hashCode must be consistent across calls. Violation scenarios: CASE 1: equals() overridden but hashCode() not — two 'equal' objects land in different buckets; containsKey/get returns false even for logically equal keys; duplicates appear in the map. CASE 2: hashCode() overridden but equals() not — objects with same hashCode in same bucket; get() then uses equals() for chain traversal; since equals() is Object == (identity), only the EXACT same reference is found; logically equal keys are treated as distinct. CASE 3: hashCode() is inconsistent (returns different values for same object state) — the entry is stored in a bucket that no longer matches after the hashCode changes. Always override hashCode when you override equals. Java records do this correctly automatically.

[ADV] Q50. What is HashMap's TreeNode structure, and how does it differ from TreeMap's Entry?

HashMap's TreeNode extends LinkedHashMap.Entry (which extends HashMap.Node) to also be a Red-Black tree node. TreeNode has: parent, left, right, prev (for doubly-linked tree structure), boolean red. This means a treeified bin in HashMap is BOTH a Red-Black tree (for O(log n) lookup) AND a doubly-linked list of nodes in insertion order (to untreeify efficiently). TreeMap's Entry is a pure Red-Black tree node with parent, left, right, color — no linked list. HashMap's TreeNode is heavier (more fields) because it needs to serve both structures. HashMap's treeified bins also maintain the original bucket-level structure — the TreeNode at position i serves as the bin head in table[i].

[ADV] Q51. Explain the memory layout difference between HashMap and EnumMap for a map with 10 entries.

HashMap: object header + Node[] table (capacity 16 or 32 with load factor) + 10 Node objects (each: header + int hash + K key ref + V value ref + Node next ref = ~40 bytes each on 64-bit JVM). Plus possible tree nodes (heavier) if treeified. Total: ~800-1000+ bytes for 10 entries. EnumMap: object header + Object[] vals (exactly N elements, where N = enum constant count, e.g. 10). No Node objects, no hash computation, no linked list overhead. A 10-enum EnumMap uses approximately one array of 10 object references = ~80-100 bytes plus headers. This is roughly 10x smaller. Additionally, EnumMap's array is a contiguous memory region — far better CPU cache utilization during iteration compared to HashMap's scattered Node heap objects.

[ADV] Q52. In a high-throughput microservice, you need a shared in-memory counter map (String -> Long) updated by hundreds of threads. What is the optimal approach?

Option 1 (ConcurrentHashMap + merge): `concMap.merge(key, 1L, Long::sum)`. Atomic per key, good for moderate concurrency. The merge fn is called under the bin lock — contention if many threads update the SAME key simultaneously. Option 2 (ConcurrentHashMap + LongAdder): `concMap.computeIfAbsent(key, k -> new LongAdder()).increment()`. LongAdder uses internal striping (Striped64) — each thread increments its own cell, `sum()` at read. MUCH better throughput under high per-key contention. Option 3 (Partition by key): use consistent hashing or `key.hashCode() % numPartitions` to assign keys to per-thread ConcurrentHashMap shards — reduces contention further. For read-mostly with infrequent writes: CopyOnWriteMap (expensive writes, O(1) lock-free reads). Recommended production choice: ConcurrentHashMap with `computeIfAbsent`.

Exercises

Part A — Coding Challenges

A1. Implement a full generic LRU cache class `LRUCache` using `LinkedHashMap`. The class must have: constructor `LRUCache(int capacity)`, `get(K key)` returning null on miss, `put(K key, V value)`, and an `isFull()` method. Verify that the eldest entry is evicted when capacity is exceeded. Write a `main()` demonstrating eviction.

A2. Write a method `Map wordFrequency(String text)` that counts word frequencies using `Map.merge()`. Handle null and empty input gracefully without throwing exceptions. The method should be case-insensitive and strip punctuation. Use only Java standard library.

A3. Implement a `MultiMap` class backed by `Map>` with the following methods: `void addToGroup(K key, V value)`, `boolean removeFromGroup(K key, V value)`, `List getGroup(K key)` (returns empty list if absent, never null), `Set keys()`, and `int totalValues()`. Use `computeIfAbsent` for `addToGroup`.

A4. Write a static method `Set findDuplicateValueKeys(Map map)` that returns the set of keys whose `Integer` values appear more than once in the map. Example: `{a:1, b:2, c:1, d:3, e:2}` should return `{a, b, c, e}`. Solve using a single stream pipeline with `groupingBy` and `filtering`.

A5. Implement a thread-safe frequency counter `ThreadSafeCounter` using `ConcurrentHashMap`. It must support: `void increment(String key)`, `void incrementBy(String key, long delta)`, `long getCount(String key)`, `Map snapshot()` (returns an unmodifiable copy), and `String topKey()` returning the key with the highest count. Use `ConcurrentHashMap.merge()` for atomic increments.

Part B — Debug & Fix

B1 – Concurrent HashMap Corruption

```
// Two threads executing simultaneously:
// Thread 1:
map.put("counter", map.getOrDefault("counter", 0) + 1);
// Thread 2:
map.put("counter", map.getOrDefault("counter", 0) + 1);
// Expected: counter = 2. Actual: counter = 1 (race condition)
```

Fix: `get()` and `put()` are separate operations — another thread can interleave between them. The two increments race and one is lost. Fix 1: use `ConcurrentHashMap.merge("counter", 1, Integer::sum)` — atomic. Fix 2: use `ConcurrentHashMap.compute("counter", (k,v) -> v==null?1:v+1)` — atomic. Fix 3: switch to `ConcurrentHashMap` for high-throughput counters.

B2 – Mutable Key Corruption

```
class Point {
    int x, y; // mutable!
    // equals/hashCode based on x and y
}
Map<Point, String> map = new HashMap<>();
Point p = new Point(1, 2);
map.put(p, "origin area");
p.x = 99; p.y = 99; // mutate after insert!
System.out.println(map.get(p)); // null – lost!
```

Fix: Mutating `p.x` and `p.y` changes `hashCode`. `HashMap` looks in bucket `hash(99,99)&(n-1)` but the entry is still in bucket `hash(1,2)&(n-1)`. Fix: make `Point` immutable (final fields, no setters). Better: use a Java record: `record Point(int x, int y) {}` — immutable by design, correct `equals/hashCode` generated automatically.

B3 – TreeMap ClassCastException

```
class Employee {
    String name; int id;
    // No Comparable, no Comparator provided
}
TreeMap<Employee, String> map = new TreeMap<>(); // No Comparator!
map.put(new Employee("Alice", 1), "Engineering");
map.put(new Employee("Bob", 2), "Marketing"); // ClassCastException here!
```

Fix: TreeMap requires keys to be Comparable OR a Comparator must be supplied. Employee does neither. The exception occurs on the second put() when TreeMap tries to compare the new key to existing keys. Fix 1: implement Comparable in Employee (e.g., compare by id). Fix 2: provide a Comparator: new TreeMap<>(Comparator.comparingInt(e -> e.id)). Fix 3: use HashMap if no ordering is needed.

B4 – ConcurrentHashMap NullPointerException

```
ConcurrentHashMap<String, String> map = new ConcurrentHashMap<>();
String key = "config";
String value = System.getProperty("missing.property"); // returns null!
map.put(key, value); // NullPointerException!
```

Fix: ConcurrentHashMap.put() throws NullPointerException if key or value is null — by design. System.getProperty returns null for missing properties. Fix: guard with null check: if (value != null) map.put(key, value); Or use a default: map.put(key, value != null ? value : "default"); Or: map.put(key, Objects.requireNonNullElse(value, "default")); Never use null values in ConcurrentHashMap — they are ambiguous under concurrency.

Part C — MCQ / Predict the Output

MC1. What is the output? Map m = new HashMap<>(); m.merge("a", 1, Integer::sum); m.merge("a", 1, Integer::sum); m.merge("a", 1, Integer::sum); System.out.println(m.get("a"));

- A) 1
- B) 2
- C) 3
- D) NullPointerException

Answer: C — merge("a", 1, Integer::sum): first call puts 1 (absent); second call: sum(1,1)=2; third: sum(2,1)=3. Result is 3.

MC2. Map m = new LinkedHashMap<>(); m.put("c",3); m.put("a",1); m.put("b",2); System.out.println(m.keySet());

- A) [a, b, c]
- B) [c, a, b]
- C) [b, a, c]
- D) Random order

Answer: B — LinkedHashMap preserves insertion order. Keys were inserted c, a, b — so iteration order is [c, a, b].

MC3. TreeMap m = new TreeMap<>(); m.put(5,"e"); m.put(1,"a"); m.put(3,"c"); System.out.println(m.floorKey(4));

- A) 5
- B) 4
- C) 3
- D) null

Answer: C — floorKey(4) returns the largest key <= 4. Keys are {1, 3, 5}. Largest <= 4 is 3.

MC4. ConcurrentHashMap m = new ConcurrentHashMap<>(); m.put("x", null); // What happens?

- A) Puts null value successfully
- B) NullPointerException
- C) IllegalArgumentException
- D) Returns null silently

Answer: B — ConcurrentHashMap.put() throws NullPointerException for null values (and null keys). This is by design.

MC5. EnumMap m = new EnumMap<>(Day.class); // Day = {MON,TUE,WED} m.put(Day.WED, "w"); m.put(Day.MON, "m"); System.out.println(m.keySet());

- A) [WED, MON]
- B) [MON, WED]
- C) [MON, TUE, WED]
- D) Random

Answer: B — EnumMap iterates in enum declaration order (ordinal order). MON=0, WED=2 → keySet() = [MON, WED].

MC6. Map m = new HashMap<>(Map.of("a",1,"b",2)); m.computeIfPresent("a", (k,v) -> null); System.out.println(m.containsKey("a"));

- A) true
- B) false
- C) NullPointerException
- D) UnsupportedOperationException

Answer: B — computeIfPresent returns null from the function — this causes the key to be REMOVED from the map. containsKey("a") returns false. Note: the outer map is mutable (created via new HashMap<>(Map.of(...))).

Quick Reference Cheat Sheet

Topic	Key Points to Remember
Map vs Collection	Map does NOT extend Collection. Holds key-value pairs, not elements. Iteration via <code>entrySet()</code> , <code>keySet()</code> , <code>values()</code>
HashMap internals	<code>Node[]</code> table, power-of-2 capacity, $\text{hash} = h \gg 16$, $\text{index} = \text{hash} \& (\text{cap}-1)$, load factor 0.75
Treeify conditions	$\text{Bin} \geq 8$ AND $\text{table.length} \geq 64 \rightarrow \text{treeify}$. $\text{Table} < 64 \rightarrow \text{resize instead}$. Untreeify at $\text{bin} < 6$
HashMap null	Allows 1 null key (hashed as 0), multiple null values. <code>ConcurrentHashMap/Hashtable</code> forbid both
Pre-size formula	<code>new HashMap<>((expectedSize / 0.75 + 1))</code> avoids resize. Guava: <code>Maps.newHashMapWithExpectedSize(n)</code>
LinkedHashMap modes	Insertion-order (default) or access-order (3rd constructor arg = true). Access-order $\rightarrow$ LRU cache
LRU cache pattern	extend <code>LinkedHashMap</code> , pass <code>accessOrder=true</code> , override <code>removeEldestEntry</code> to return <code>size() > capacity</code>
TreeMap	Red-Black tree, $O(\log n)$. Keys must be <code>Comparable</code> or supply <code>Comparator</code> . No null keys by default
NavigableMap ops	<code>floor(<=)</code> , <code>ceiling(>=)</code> , <code>lower(<)</code> , <code>higher(>)</code> . <code>headMap/tailMap/subMap</code> return LIVE views
ConcurrentHashMap	Per-bin locking (CAS for empty bins, synchronized on bin head). No null key/value. Weakly consistent iterator
CHM atomic ops	<code>compute/merge/computeIfAbsent</code> are atomic per key. <code>size()/mappingCount()</code> are approximate
EnumMap	Array indexed by ordinal. $O(1)$, no null key. Iterates in enum declaration order. Most efficient for enum keys
WeakHashMap	Keys are <code>WeakReferences</code> . GC can reclaim keys. Entries purged lazily via <code>ReferenceQueue</code> . Use for metadata caches
IdentityHashMap	Uses <code>==</code> for key comparison, <code>System.identityHashCode()</code> . Use for object graph traversal, serialization
merge() idiom	<code>map.merge(k, 1, Integer::sum)</code> — frequency counting. Absent: put value; present: apply function; null result: remove
computeIfAbsent	<code>map.computeIfAbsent(k, x -> new ArrayList<>()).add(v)</code> — group-by / multi-map pattern. Lazy and atomic
Map.of() rules	Immutable, no null key/value, no duplicate keys (IAE). Order randomized per JVM run (anti-hash-flooding)
Thread-safe choices	<code>ConcurrentHashMap</code> (high concurrency) > <code>synchronizedMap</code> (simple wrap) > <code>Hashtable</code> (legacy, avoid)

End of Sub-Chapter 1.4 — Collections: Map Implementations

Next: [Sub-Chapter 1.5 — Queue, Deque & BlockingQueue](#)